

Chapter 3 : In Class Exercise

1. Determine whether **goal-driven** or **data-driven search** would be preferable for solving each of the following problems. Justify your answer.

- a) Diagnosing mechanical problems in an automobile *goal-driven*
- b) You have met a person who claims to be your distant cousin, with a common ancestor named John Doe. You would like to verify her claim. *goal-driven*
- c) Another person claims to be your distant cousin. He does not know the common ancestor's name but knows that it was no more than eight generations back. You would like to either find this ancestor or determine that she did not exist. *data-driven*
- d) A theorem prover for plane geometry. *goal-driven*
- e) A program for examining sonar readings and interpreting them, such as telling a large submarine from a small submarine from a whale from a school of fish. *data-driven*
- f) An expert system that will help a human classify plants by species, genus, etc. *data-driven*

2. Apply the backtrack algorithm on the graph in **Figure 1**. Begin from state A. Keep track of the successive values of NSL, SL, CS, etc.

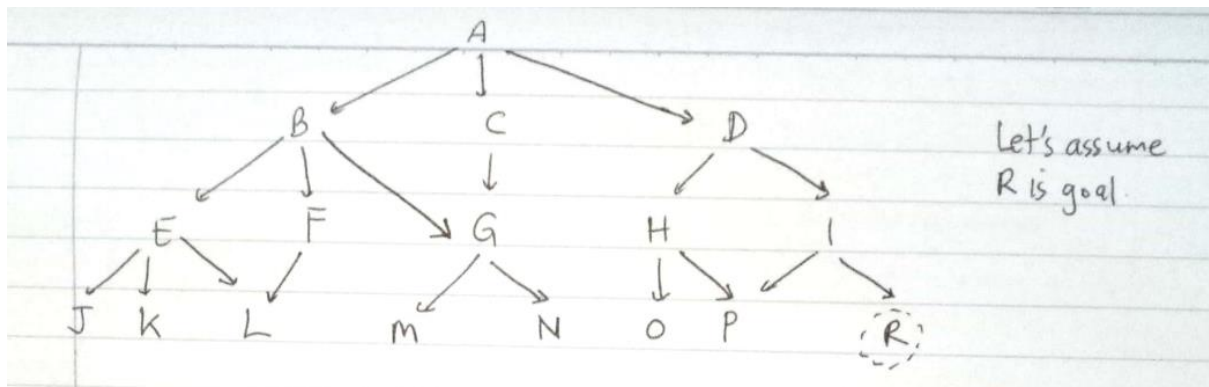


Figure 1

Initialize : SL=[A] ; NSL=[A] ; DE=[] ; CS=A;

	CS	SL	NSL	DE
0	A	[A]	[A]	[]
1	B	[BA]	[BCDA]	[]
2	E	[EBA]	[EFGBCDA]	[]
3	J	[JEBA]	[JKEFGBCDA]	[]
4	K	[KEBA]	[KLEFGBCDA]	[J]
5	L	[LEBA]	[LEFGBCDA]	[KJ]
6	F	[FBA]	[FGBBCDA]	[ELKJ]
7	G	[GBA]	[GBCDA]	[FELKJ]
8	M	[MGBA]	[MNGBCDA]	[FELKJ]
9	N	[NGBA]	[NGBBCDA]	[MFELKJ]
10	C	[CA]	[CDA]	[BGNMFELKJ]
11	D	[DA]	[DA]	[CBGNMFELKJ]
12	H	[HDA]	[HIDA]	[CBGNMFELKJ]
13	O	[OHDA]	[OPHIDA]	[CBGNMFELKJ]
14	P	[PHDA]	[PHIDA]	[OCBGNMFELKJ]
15	I	[IDA]	[IDA]	[HPOCBGNMFELKJ]
16	R	[RIDA]	[RIDA]	[HPOCBGNMFELKJ]

ARTIFICIAL INTELLIGENCE

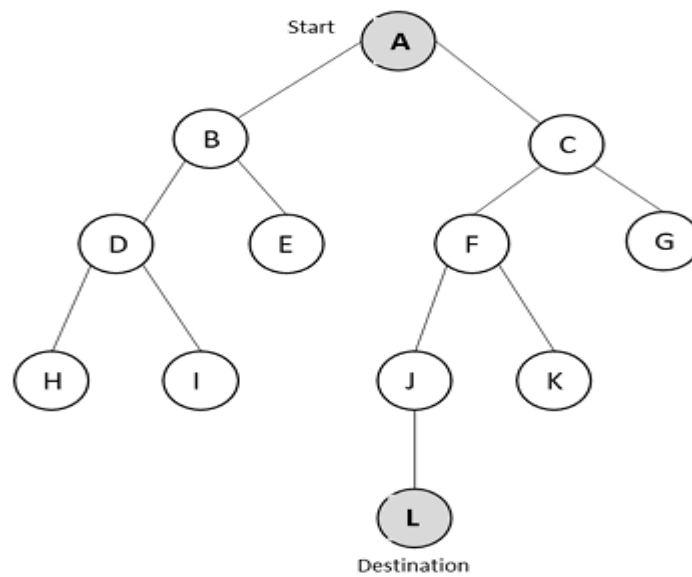


Figure 3.0

7. You are applying Depth-First Search (DFS) to find a path from node A to node L (see Figure 3.0). Initialize the open list, closed list, and start the traversal from node A

a. Trace the search tree until you reach the destination (node L). Indicate the open list, closed list, and the current node at each step/iteration.

- | | |
|---|------------------|
| 1. open = [A] ; closed = [] | Current node = A |
| 2. open = [B, C] ; closed = [A] | = B |
| 3. open = [D, E, C] ; closed = [B, A] | = D |
| 4. open = [H, I, E, C] ; closed = [D, B, A] | = H |
| 5. open = [I, E, C] ; closed = [H, D, B, A] | = I |
| 6. open = [E, C] ; closed = [I, H, D, B, A] | = E |
| 7. open = [C] ; closed = [E, I, H, D, B, A] | = C |
| 8. open = [F, G] ; closed = [C, E, I, H, D, B, A] | = F |
| 9. open = [J, K, G] ; closed = [F, C, E, I, H, D, B, A] | = J |
| 10. open = [L, K, G] ; closed = [J, F, C, E, I, H, D, B, A] | = L |

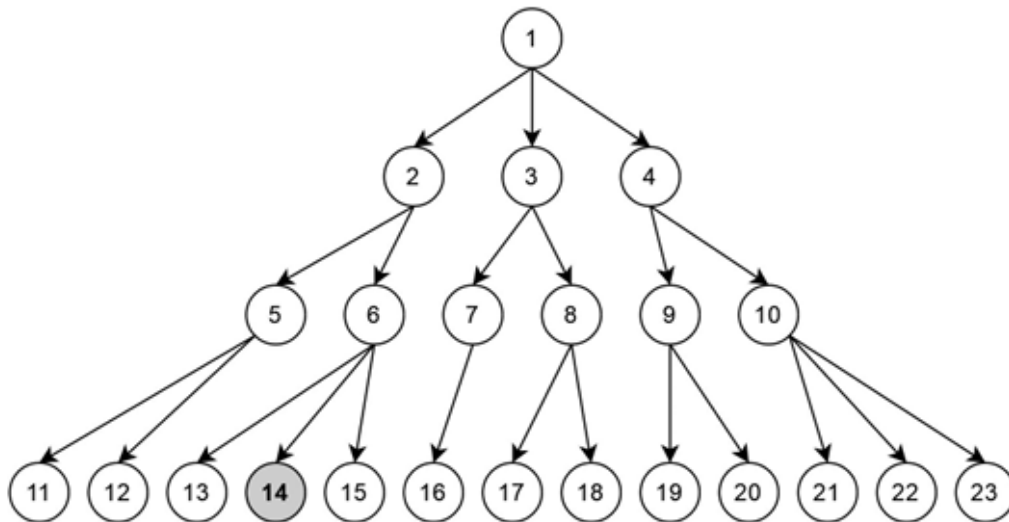


Figure 2.0

6. a. Perform Breadth-First Search on the state space graph in Figure 2.0 above. List down the order of nodes visited from starting node (Node 1) to the goal (Node 14) in a table as shown in Table 1.0.

* You may add more row if necessary.

(13 marks)

Table 1.0

Iteration	Open List	Closed List
0	1	[]
1	[2, 3, 4]	[1]
2	[3, 4, 5, 6]	[2, 1]
3	[4, 5, 6, 7, 8]	[3, 2, 1]
4	[5, 6, 7, 8, 9, 10]	[4, 3, 2, 1]
5	[6, 7, 8, 9, 10, 11, 12]	[5, 4, 3, 2, 1]
6	[7, 8, 9, 10, 11, 12, 13, 14]	[6, 5, 4, 3, 2, 1]
7	[8, 9, 10, 11, 12, 13, 14]	[7, 6, 5, 4, 3, 2, 1]
8	[9, 10, 11, 12, 13, 14]	[8, 7, 6, 5, 4, 3, 2, 1]
9	[10, 11, 12, 13, 14]	[9, 8, 7, 6, 5, 4, 3, 2, 1]
10	[11, 12, 13, 14]	[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
11	[12, 13, 14]	[11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
12	[13, 14]	[12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
13	[14]	[13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]